

Atividade: Identifique e justifique esta identificação de modelos de arquiteturas de sistemas distribuídos

1. Uma clínica pequena desenvolve um sistema de agendamento e prontuário eletrônico usado apenas em 3 unidades, com poucos usuários simultâneos e regras de negócio estáveis. A equipe é enxuta e júnior, então optam por um monolito web em 3 camadas (frontend, backend, banco relacional) para ter desenvolvimento simples, deploy único e debugging fácil.

- A equipe resolveu utilizar um monolito em 3 camadas, por serem uma equipe pequena e formada apenas por juniores, um código unificado e uma única ferramenta facilitam muito a curva de aprendizado da equipe. A arquitetura também facilita os testes, pois tudo roda em um único processo e o debugging local é imediato. Oferece também uma divisão clara entre as camadas, sendo de apresentação, lógica de negócio e persistência, assim facilitando a manutenção inicial e encoraja separation of concerns.

2. Um banco grande precisa integrar sistemas de cartão, crédito, câmbio e internet banking, muitos deles legados. Eles expõem serviços coarse-grained como “Serviço de Clientes”, “Serviço de Contas” e “Serviço de Pagamentos” via um barramento corporativo, permitindo que vários canais (web, mobile, caixas eletrônicos) utilizem os mesmos serviços sem reescrever lógica crítica.

- O banco usa arquitetura Coarse-grained in Service-Oriented Architecture (SOA). Oferecendo reusabilidade de serviços em múltiplas aplicações (web, mobile, caixas eletrônicos), assim, reduzindo duplicação de lógica. Através de contratos de serviço bem definidos é possível integrar os múltiplos sistemas legados, cada serviço pode ser escalonado de forma independente, permitindo alocar recursos conforme a demanda real de cada módulo. A independência entre serviços (granularidade) também permite que analistas de negócio entendam a finalidade de cada serviço e discutam requisitos com desenvolvedores de forma mais acessível.

3. Uma plataforma de streaming global separa o sistema em dezenas de microserviços: catálogo, recomendação, billing, playback, usuários, notificações, etc. Cada time é responsável por um conjunto pequeno de serviços, faz deploy independente e escala só os serviços mais exigidos, como recomendação e playback, garantindo resiliência e evolução rápida em larga escala.

- A plataforma utiliza a arquitetura Microservices, assim cada microserviço consegue escalar sozinho conforme sua demanda de forma independente, sem interferir em outros serviços que a aplicação utilize, equipes podem deployar serviços sem coordenar com outras equipes. À isolamento de falhas, se um serviço parar de funcionar, os outros continuam operando! Com essa arquitetura a plataforma poderá organizar seu time por domínio de acordo com a área de negócio.

4. Uma startup de IoT coleta dados de sensores em milhares de dispositivos espalhados pelo país. Eventos de leitura chegam em um stream, disparam funções serverless para validação, agregação em tempo real e geração de alertas. Outros microserviços consomem esses eventos para dashboards e relatórios. A carga é altamente variável, então a escalabilidade automática e o modelo pay-per-use evitam pagar por servidores ociosos.

- A startup utiliza a arquitetura Event-Driven Architecture (EDA) combinada com Serverless (FaaS). Os sensores produzem eventos contínuos que alimentam um stream central, disparando funções serverless de forma reativa para validação, agregação e geração de alertas. Como a carga é altamente variável, a escalabilidade automática garante que novas instâncias sejam criadas sob demanda e encerradas quando ociosas, seguindo o modelo pay-per-use e evitando custos desnecessários. O desacoplamento entre produtores (sensores) e consumidores (funções e microserviços) permite que cada parte do pipeline evolua de forma independente, sem que uma falha em um componente comprometa os demais. Isso reduz também a carga operacional da equipe, que não precisa gerenciar servidores, focando apenas no domínio do negócio.